

# Sparse LLM Planning for Leader–Follower UAV Swarm Formation

1<sup>st</sup> Shiping Zhao

*School of Electrical Engineering and Automation  
Anhui University  
Hefei, China*

3<sup>rd</sup> Qingzhong Gan

*Shanghai Aerospace Control Technology Institute  
Shanghai, China*

2<sup>nd</sup> Jinglei Liu

*Beijing Institute of Space Mechanics and Electricity  
China Academy of Space Technology  
Beijing, China*

4<sup>th</sup> Liang Zhang\*

*School of Electrical Engineering and Automation  
Anhui University  
Hefei, China*

**Abstract**—Natural-language interfaces can simplify UAV swarm formation planning, but asking a large language model (LLM) to return one target coordinate per UAV couples structured-output length to swarm size. This paper presents a sparse leader–follower formation framework that assigns the LLM only the contour key points needed to describe a shape. A local controller selects leaders by distance, guides them to the key points, and uses leader attraction and local repulsion to distribute followers along the contour. Decomposable shapes are processed through parallel subtasks with feedback correction, while the same motion controller is retained across formation changes. Simulation comparisons cover circles, five-pointed stars, and the disconnected text “HI,” with up to 170 UAVs. The reported curves show more stable completion at larger swarm sizes than direct per-UAV target generation; the advantage over an SDF-based planner is more pronounced for complex contours than for circles. At 170 UAVs, the proposed method achieves approximately 73% and 80% success for the star and “HI,” respectively. These exploratory results support reducing the dependence of LLM target output on swarm population, while leaving the individual effects of sparsity, parallelism, and feedback to future ablation studies.

**Index Terms**—UAV swarms, large language models, formation control, sparse planning, leader–follower control.

## I. INTRODUCTION

Aerial swarm autonomy combines task allocation, trajectory generation, sensing, and coordinated flight [1]. Local interaction models provide a foundation: Reynolds’ Boids rules generate collective motion from separation, alignment, and cohesion [2], while flocking-control theory analyzes coordinated dynamics under explicit assumptions [3]. More recent systems integrate onboard perception and trajectory optimization to navigate micro-UAV swarms through cluttered environments [4]. Related work also considers distributed positioning

coverage in multi-robot systems [5], safety-critical UAV trajectory tracking [6], redundant-SINS fault diagnosis and initial alignment [7], and relay-assisted recursive and set-membership state estimation [8], [9]. These advances address execution, but a user still needs a way to specify the desired collective geometry. Letters, irregular contours, and disconnected shapes can require additional geometric modeling before a controller can act.

Large language models (LLMs) offer several interfaces between instructions and robot execution. SayCan grounds language-based skill selection in learned affordances [10]. Code as Policies generates executable robot programs [11], and ProgPrompt uses program-like descriptions of actions, objects, and example plans to condition task planning [12]. VoxPoser translates language and visual observations into spatial value maps for model-based manipulation planning [13]. These interfaces separate semantic specification from numerical execution, but leave the geometric representation for large UAV formations open.

For multi-robot tasks, SMART-LLM organizes task decomposition, coalition formation, and allocation [14]. RoCo combines robot dialogue with waypoint planning and environment feedback [15], whereas LiP-LLM couples language-generated skill dependencies with linear-programming allocation [16]. These systems address coordination and task structure, motivating a division of work between semantic reasoning and local planning.

Closer to swarm formation, FlockGPT generates signed distance function (SDF) geometry code and then constructs and optimizes target point clouds locally [17]. Its compact geometric representation already weakens the dependence of LLM output on robot count. GenSwarm generates, reviews, and deploys multi-robot code policies through task constraints, skills, and skill graphs [18]. SwarmGPT combines language-based drone choreography with a predictive safety filter [19]. Table I contrasts the main output representations. A recent perspective distinguishes foundation models as designers of

\* Correspondence: Liang Zhang (liangzhang@ahu.edu.cn)

This work was supported in part by the National Natural Science Foundation of China under grant No. 62303002, in part by the State Key Laboratory of Micro-Spacecraft Rapid Design and Intelligent Cluster under grant No. MS01240110, and in part by the Key Program of Natural Science Foundation of Anhui Higher Education Institutions of China under grant No. 2022AH050068.

TABLE I  
REPRESENTATIONS BETWEEN LANGUAGE AND LOCAL EXECUTION

| Method                | LLM output                             | Local execution                           |
|-----------------------|----------------------------------------|-------------------------------------------|
| Per-UAV targets       | One coordinate per UAV                 | Assignment and target tracking            |
| FlockGPT [17]         | SDF geometry code                      | Sampling, point optimization, and control |
| GenSwarm [18]         | Constraints, skills, and code policies | Deployment and skill execution            |
| Sparse anchors (ours) | Contour key points                     | Leader selection and follower completion  |

controllers and plans from models acting as swarm operators [20], emphasizing the boundary between semantic output and local control.

The present work focuses on the size of the geometric output. A direct interface asks the LLM for one target coordinate per UAV, followed by local assignment and tracking. Although inference remains outside the continuous control loop, the number of output records still grows with swarm population. Larger requests can increase generation time and expose the pipeline to incomplete or malformed output. Compact geometric code avoids this dense representation, but its usefulness depends on correct expression and composition of the requested shape.

We instead use a small set of explicit contour key points. The LLM describes the shape at selected anchors; distance-based leader selection and local follower interactions expand that description to the swarm. For a fixed contour and anchor set, adding followers need not add LLM-generated target records. Decomposable shapes can be planned through concurrent subproblems and feedback correction, and successive static formations reuse the same motion controller.

The contributions are threefold. First, we formulate a sparse target interface separating the number of semantic anchors from the number of UAVs. Second, we integrate leader assignment, a Boids-inspired follower controller, and parallel planning with feedback into a reusable static-formation process. Third, circle, star, and “HI” simulations and an A–H–U switching example examine how the combined framework behaves across population sizes and contour complexities. The evidence does not isolate individual component gains or establish a universal advantage over SDF planning.

## II. PROBLEM FORMULATION

### A. Static Formation Task

Consider  $N$  UAVs in a plane, with position  $\mathbf{p}_i \in \mathbb{R}^2$  and velocity  $\mathbf{v}_i \in \mathbb{R}^2$ . A natural-language instruction specifies the desired contour, its location, and its scale. The output of sparse planning is

$$\mathcal{Q} = \{\mathbf{q}_1, \dots, \mathbf{q}_K\}, \quad K \leq N, \quad (1)$$

where the  $K$  anchors describe the target geometry. A subset  $\mathcal{L}$  of  $K$  UAVs serves as leaders, and  $\mathcal{F}$  contains the remaining followers. Each leader receives a distinct anchor; followers obtain guidance locally. Here, “leader” is a temporary control

role assigned for a formation, rather than a fixed hardware category.

The design objective is to retain a recognizable spatial formation while reducing the LLM’s structured target output. A dense interface returns  $N$  coordinate records, whereas the sparse interface returns  $K$ . Thus, its coordinate-record ratio is  $K/N$  for a fixed task. This is a representation-level comparison: actual token counts also depend on the schema, reasoning text, prompt state, and feedback messages. No measured token reduction is inferred from this ratio.

The number of anchors depends on contour complexity and the desired geometric detail. A shape with additional corners or disconnected components may require more anchors even if  $N$  is unchanged. Conversely, increasing the population for the same contour can primarily increase  $|\mathcal{F}|$ . The intended regime is therefore  $K \ll N$ , rather than a claim that a fixed anchor count can describe every shape with arbitrary accuracy.

## III. SPARSE LEADER–FOLLOWER FORMATION

### A. Planning and Local Assignment

Fig. 1 summarizes the planning–execution loop. The language layer parses the requested geometry, estimates the number of anchors needed to describe it, and returns their coordinates. Anchor estimation is heuristic; the framework does not solve or certify a minimum-cardinality contour approximation. Geometric target generation is separated from Boids-parameter adjustment so that these distinct reasoning tasks can be organized independently.

For a target with separable components, the planner divides it into subproblems and processes them concurrently. The reported implementation favors subproblems with approximately 20 or fewer key points, guided by its preliminary single-thread tests. This is an operating heuristic for that configuration, rather than an LLM capacity limit. Subproblem outputs are assembled into the global anchor set before local execution. Disconnected text is a natural use case because its components can often be described separately.

The local assignment stage selects distinct UAVs using their distances to the anchors. Writing  $a : \mathcal{L} \rightarrow \{1, \dots, K\}$  for the one-to-one association, leader  $i$  tracks  $\mathbf{q}_{a(i)}$ . The remaining UAVs become followers. The method requires a distance-based association, but does not rely on a claim of globally optimal assignment. When a new shape is requested, roles and associations can be updated through the same procedure.

### B. Local Motion Controller

The velocity increment combines attraction, separation, and optional alignment. Let  $\mathcal{N}_i$  denote the local neighbors of UAV  $i$ , and let  $d_{ij} = \|\mathbf{p}_i - \mathbf{p}_j\|$ . The separation and alignment terms are

$$\mathbf{s}_i = w_{\text{rep}} \sum_{\substack{j \in \mathcal{N}_i \\ 0 < d_{ij} < r_{\text{rep}}}} (r_{\text{rep}} - d_{ij}) \frac{\mathbf{p}_i - \mathbf{p}_j}{d_{ij}}, \quad (2)$$

$$\mathbf{h}_i = w_{\text{ali}} \left( \frac{1}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} \mathbf{v}_j - \mathbf{v}_i \right), \quad (3)$$

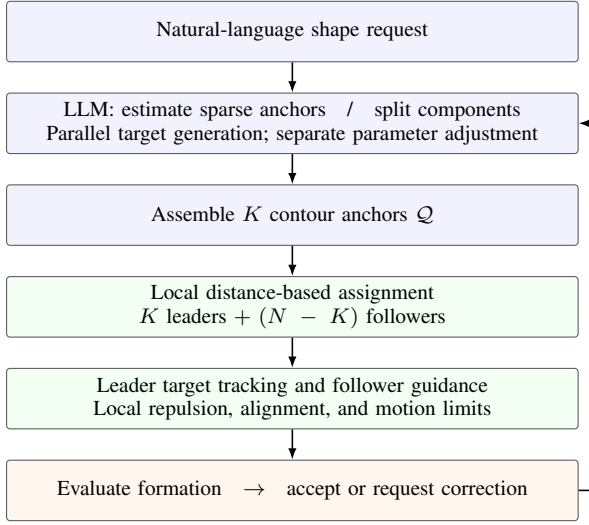


Fig. 1. Sparse semantic planning and local swarm execution. Feedback revises the anchor description or parameters; individual motion remains a local computation.

with  $\mathbf{h}_i = \mathbf{0}$  for an empty neighborhood. Here  $r_{\text{rep}}$  is the repulsion range. Setting  $w_{\text{ali}} = 0$  disables velocity alignment when it is unnecessary for target tracking. Equation (2) assumes distinct positions; exact overlaps require a separate numerical tie rule.

For a leader, the attraction term is

$$\mathbf{a}_i = w_{\text{att}}(\mathbf{q}_{a(i)} - \mathbf{p}_i), \quad i \in \mathcal{L}. \quad (4)$$

A follower  $i$  is associated with a leader  $\ell_i$  and also guided by the nearest other leader  $m_i$ . Its two leader-guidance terms are

$$\mathbf{g}_i = w_m(\mathbf{p}_{\ell_i} - \mathbf{p}_i) + w_n(\mathbf{p}_{m_i} - \mathbf{p}_i), \quad i \in \mathcal{F}, \quad (5)$$

where the associated leader receives the stronger weight,  $w_m > w_n \geq 0$ . The second term uses the other leader's position. Attraction draws followers toward the region between leaders, while repulsion discourages local clustering. This mechanism supplies follower guidance without an additional LLM-generated target for each follower.

Using  $\mathbf{u}_i = \mathbf{a}_i$  for leaders and  $\mathbf{u}_i = \mathbf{a}_i^{\text{loc}} + \mathbf{g}_i$  for followers, the role-dependent update is

$$\Delta \mathbf{v}_i = \mathbf{u}_i + \mathbf{s}_i + \mathbf{h}_i. \quad (6)$$

Here  $\mathbf{a}_i^{\text{loc}}$  denotes any additional follower goal-attraction term supplied locally. The experiment description mentions this term but does not specify its target-generation rule, so the exact follower controller cannot be independently recovered from the available record. Define norm saturation by  $\text{sat}_c(\mathbf{z}) = \mathbf{z} \min\{1, c/\|\mathbf{z}\|\}$  for nonzero  $\mathbf{z}$ , and  $\text{sat}_c(\mathbf{0}) = \mathbf{0}$ . The motion update can then be written without divisions by zero as

$$\mathbf{v}_i^{t+1} = \mathbf{v}_i^t + \text{sat}_{\Delta v_{\text{max}}}(\Delta \mathbf{v}_i^t), \quad (7)$$

$$\mathbf{p}_i^{t+1} = \mathbf{p}_i^t + \Delta t \text{sat}_{v_{\text{max}}}(\mathbf{v}_i^{t+1}). \quad (8)$$

Here,  $t$  denotes the discrete time index,  $\Delta t$  is the integration time step,  $\Delta v_{\text{max}}$  is the maximum allowed velocity increment

per step, and  $v_{\text{max}}$  is the maximum UAV speed. The increment is limited before velocity storage, and the speed used for displacement is limited separately. These numerical bounds do not certify multirotor actuation feasibility or collision-free trajectories. Contour completion is evaluated empirically.

### C. Feedback and Computation Scaling

Splitting a shape can reduce the context available to each subproblem. The assembled output may contain gaps, misplaced components, or missing contour details. After initial execution, the system evaluates completion and sends deficient-region information back to the LLM for supplementary anchors or parameter adjustments. The same controller executes the corrected description. The available experiments evaluate this full process together with sparse planning and parallel organization.

Let  $T_j$  denote the inference time of subproblem  $j$  and let  $S$  be the number of concurrent subproblems. With sufficient serving concurrency, the planning-time contribution is approximately

$$T_{\text{plan}} \approx T_{\text{split}} + \max_{1 \leq j \leq S} T_j + T_{\text{merge}}. \quad (9)$$

Feedback and local execution add further time. Equation (9) describes the intended scheduling structure, not a measured speedup law; queueing, shared model resources, and retries can reduce the benefit. It also explains why splitting a small task can be slower than a single direct request.

Sparse output does not make the entire system independent of  $N$ . Leader selection, neighborhood processing, and motion updates remain local population-dependent computations. A direct all-pairs neighborhood calculation can still require  $O(N^2)$  distance checks per step. The principal architectural saving is in LLM-generated target records, while total runtime depends on both the semantic planner and the local implementation.

## IV. EXPERIMENTAL EVALUATION

### A. Tasks, Comparators, and Evidence Scope

The evaluation uses planar UAV swarm simulations with a common local Boids controller and motion constraints across the compared static tasks. The main comparisons use the model configuration identified in the experiment record as deepseek-v4-flash. They compare direct single-thread per-UAV targets, an SDF-based FlockGPT implementation, and the proposed multithreaded sparse method. Results concern these evaluated configurations, rather than every possible implementation of the respective methods.

Table II lists the three shapes. Their population sweep is  $N \in \{10, 20, 30, 40, 50, 60, 90, 130, 170\}$ . The circle tests a compact analytic contour, the star introduces corners and multiple segments, and “HI” introduces disconnected components. This separates population growth from changes in geometric complexity at a descriptive level.

The experiment record provides aggregate plots, but not trial-level data, repeat counts, random seeds, prompts, decoding parameters, the complete controller settings, or a precise

TABLE II  
STATIC FORMATION TASKS AND EVALUATION MEASURES

| Target | Geometric request                       | Quality measure            |
|--------|-----------------------------------------|----------------------------|
| Circle | Center (0, 0); radius 15 m              | Continuous geometric score |
| Star   | Center (0, 0); outer scale about 15 m   | Human binary completion    |
| “HI”   | Centered region; side length about 15 m | Human binary completion    |

task-termination rule. The comparative plots also do not define their error-bar convention. Accordingly, we interpret trends and approximate plotted values without significance tests or confidence claims. The complex-shape time panels explicitly average successful runs, which must be distinguished from unconditional completion time. The plotted model names are retained as experimental identifiers; no general context-window or model-capability conclusion follows from them.

### B. Formation Quality Measures

For the circle, let  $\mathbf{o}$  be the desired center and  $R$  the desired radius. Radial error and centroid offset are

$$e_r = \frac{1}{N} \sum_{i=1}^N \left| \|\mathbf{p}_i - \mathbf{o}\| - R \right|, \quad (10)$$

$$d_c = \left\| \frac{1}{N} \sum_{i=1}^N \mathbf{p}_i - \mathbf{o} \right\|. \quad (11)$$

Sort the polar angles about  $\mathbf{o}$  as  $\theta_1 \leq \dots \leq \theta_N$  and define the cyclic gaps

$$\delta_i = \begin{cases} \theta_{i+1} - \theta_i, & i < N, \\ 2\pi + \theta_1 - \theta_N, & i = N. \end{cases} \quad (12)$$

Their deviation from uniform spacing is

$$\sigma_\theta = \sqrt{\frac{1}{N} \sum_{i=1}^N \left( \delta_i - \frac{2\pi}{N} \right)^2}. \quad (13)$$

The reported score combines exponential penalties:

$$C = w_r e^{-e_r/\ell_0} + w_\theta e^{-\sigma_\theta/\theta_0} + w_c e^{-d_c/\ell_0}, \quad (14)$$

where the nonnegative weights sum to one. Here  $\ell_0 = 1$  m and  $\theta_0 = 1$  rad make explicit the units implicit in the reported exponentials. This rewriting preserves their numerical convention; it is not a new normalization fitted to the curves. Numerical weight values are not available in the experiment record, so the aggregate score cannot be independently recomputed from the present material.

For the star and “HI,” a completed trial is assigned 1 and an incomplete trial 0. The plots report the percentage of successful trials. These human judgments assess recognizable formation completion, rather than the continuous score in (14). The two quality measures are not interchangeable, and a score of 0.8 for a circle should not be interpreted as an 80% success rate.

### C. Circle: Population Scaling with Simple Geometry

The top row of Fig. 2 shows that all methods have comparable completion scores at moderate populations. As  $N$  increases, the dense baseline deteriorates more clearly: its score is approximately 0.44 at  $N = 170$ , compared with 0.62 for the proposed method and 0.58 for FlockGPT. The proposed method maintains a relatively flat score curve, although the absolute values do not indicate perfect geometric completion.

The corresponding time curves reveal a tradeoff. Direct per-UAV planning is fast for the smallest swarm, where the request is short and no anchor-estimation or subtask-management overhead is needed. At  $N = 170$ , its reported time is approximately 53 s, compared with 31 s for the proposed method and 29 s for FlockGPT. A circle has a compact SDF representation, so FlockGPT remains competitive and slightly faster in the large-population example. The result supports sparse planning over dense coordinate output in this setting, but not a general timing advantage over SDF geometry.

### D. Complex Contours: Star and Disconnected Text

For the star, the proposed method retains a higher success rate at the larger plotted populations ( $N \geq 50$ ). At  $N = 170$ , its success is approximately 73%, versus 46% for FlockGPT and 42% for direct per-UAV targets. The mean successful completion times are approximately 72, 97, and 170 s, respectively. The proposed method’s time curve changes less with population than the dense baseline, consistent with transferring the additional follower computation to the local controller.

The disconnected “HI” task shows a similar pattern. At  $N = 170$ , the proposed method achieves about 80% success, compared with 54% for FlockGPT and 48% for direct per-UAV targets. Their mean successful completion times are approximately 134, 280, and 233 s. These values are read from the aggregate curves and are rounded accordingly. At small populations, the direct method can still finish successful trials more quickly; the relative benefit grows as the dense output becomes larger.

The shape dependence is consistent with the different intermediate representations. Explicit key points describe corners and separated components without requiring a correct composition of SDF primitives. This offers a plausible explanation for the complex-shape results, but the experiments do not include a systematic classification of SDF code errors. Nor do they isolate the contributions of parallelism, anchor sparsity, or feedback correction. The curves establish an observed comparison of the complete pipelines.

Success-conditioned timing should be considered together with success rate. A method can have a low reported mean time if difficult trials fail and are excluded. Conversely, higher success and lower successful-run time are complementary outcomes, but still do not determine the expected time under retries. That quantity requires failure durations, stopping rules, and retry policies that are not available here.

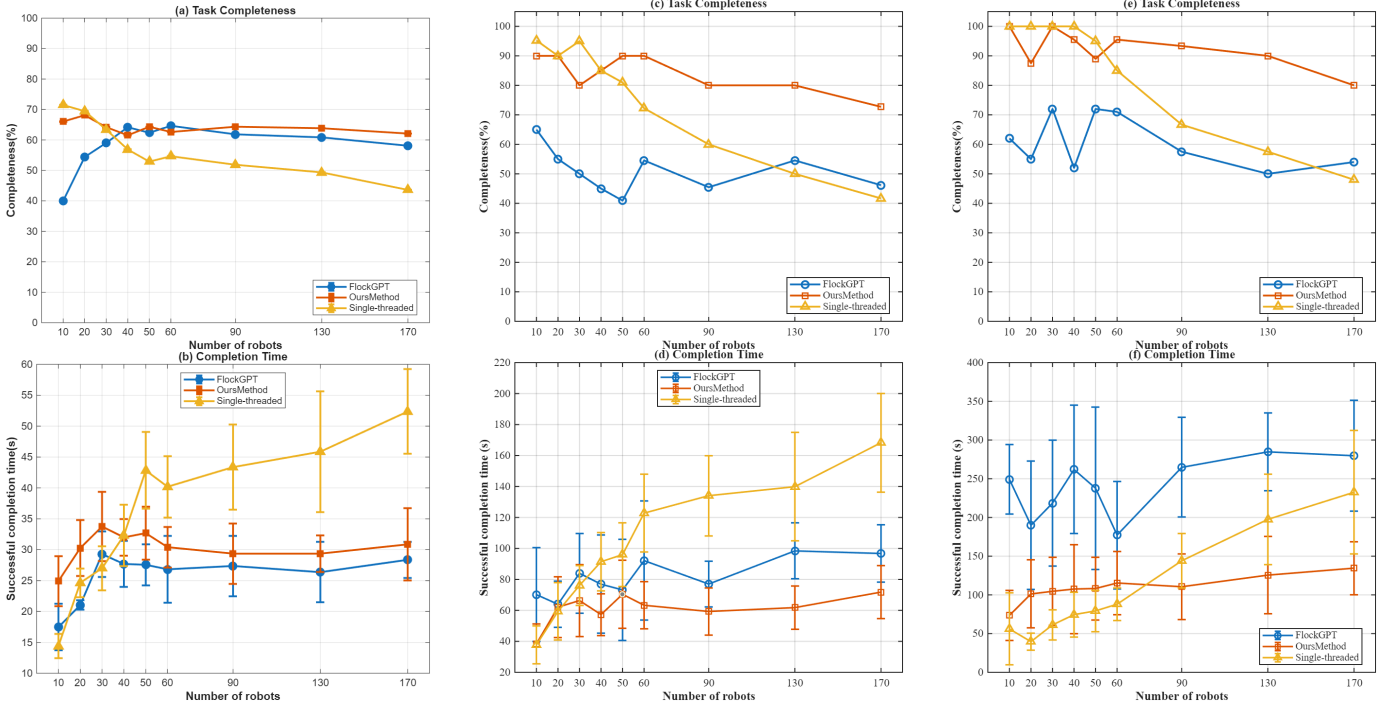


Fig. 2. Comparison of formation tasks for the circle (a-b), five-pointed star (c-d), and “HI” (e-f). The top row shows the geometric completion score for the circle and the success rates for the star and “HI” tasks; the bottom row shows the corresponding average completion times. In the legend, “OursMethod” denotes the proposed method, “Single-thread” denotes the direct per-UAV target generation method, and “Number of robots” indicates the total number of UAVs involved in each task.

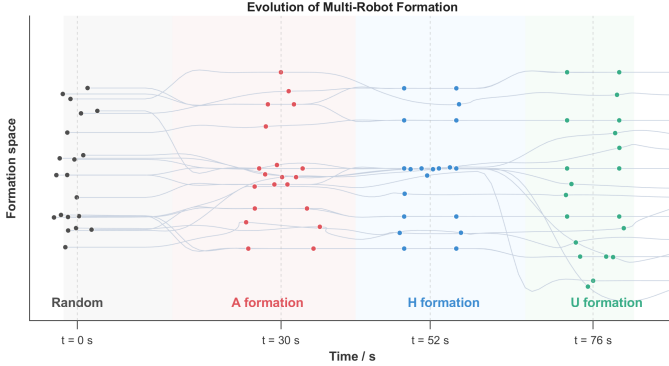


Fig. 3. Continuous switching from a random initial state to A, H, and U. The same local formation procedure is reused after updating the sparse targets. Times indicate the displayed sequence rather than a controlled timing comparison.

### E. Reuse Across Static Formation Changes

The A–H–U example in Fig. 3 tests whether one controller can execute different sparse target descriptions in sequence. Starting from a random distribution, the swarm is asked to form “A” within a centered 1.2 m region. The LLM returns key points after approximately 18 s, and the formation is reported stable around 28 s; the displayed A snapshot is at 30 s. The subsequent H and U formations are reached at approximately 52 and 76 s.

Only the requested sparse target set changes; leader assignment and local motion processing retain the same structure.

This supports reuse across the illustrated static formations. It does not establish an independent switching-speed improvement, because each response includes shape-dependent inference and local settling. Dynamic object transport and manipulation are outside the scope of this paper.

## V. DISCUSSION AND LIMITATIONS

The central result is a reduction in the dependence of semantic target output on swarm population. For a fixed shape and anchor count, an additional UAV can be incorporated as a follower without another LLM-generated coordinate. The experiments suggest that this allocation is useful for larger populations and contours that are inconvenient to express through geometric code. Small populations and simple analytic shapes can favor shorter direct pipelines or SDF planning.

Several limitations constrain this conclusion. First, the evidence is simulation-based. Planar target tracking, local repulsion, and velocity limits do not establish flight safety, communication robustness, or hardware feasibility. No stability or collision-free guarantee is derived for the leader–follower controller. Second, anchor selection and follower association are heuristic. A nearest leader may belong to a different disconnected component, and two attractive terms alone do not enforce contour topology or uniform arc-length spacing. More explicit adjacency and shape-error constraints would be needed for a guaranteed contour-completion method.

Third, the experiment record lacks the detail required for an independent numerical reproduction, and the complex-

shape scoring is subjective. Future evaluation should archive prompts, model settings, exact controller parameters, trial counts, stopping criteria, and per-trial records. Geometric contour distances and independent-rater agreement would complement the current success judgments. Finally, controlled ablations should separate sparse anchors, concurrent requests, and feedback correction, while timing should include failed trials and serving overhead. These measurements are needed before attributing a particular fraction of the observed improvement to any one component.

## VI. CONCLUSION

This paper presented a sparse language-to-formation interface in which an LLM generates contour anchors and a local leader-follower controller expands them to a UAV swarm. Parallel subtask planning and feedback correction support decomposable targets, and the same controller can be reused across successive static shapes. Circle, star, and “HI” simulations show more stable large-population completion than direct per-UAV target generation, with a stronger advantage over the evaluated SDF-based planner for complex contours. The supported conclusion is architectural and empirical: limiting semantic target output can reduce its coupling to swarm size. Reproducible trials, component ablations, topology-aware guidance, and hardware evaluation remain necessary to characterize the method more fully.

## REFERENCES

- [1] S.-J. Chung, A. A. Paranjape, P. Dames, S. Shen, and V. Kumar, “A survey on aerial swarm robotics,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 837–855, 2018, doi: 10.1109/TRO.2018.2857475.
- [2] C. W. Reynolds, “Flocks, herds and schools: A distributed behavioral model,” *ACM SIGGRAPH Computer Graphics*, vol. 21, no. 4, pp. 25–34, 1987, doi: 10.1145/37402.37406.
- [3] R. Olfati-Saber, “Flocking for multi-agent dynamic systems: Algorithms and theory,” *IEEE Transactions on Automatic Control*, vol. 51, no. 3, pp. 401–420, 2006, doi: 10.1109/TAC.2005.864190.
- [4] X. Zhou, X. Wen, Z. Wang, Y. Gao, H. Li, Q. Wang, T. Yang, H. Lu, Y. Cao, C. Xu, and F. Gao, “Swarm of micro flying robots in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm5954, 2022, doi: 10.1126/scirobotics.abm5954.
- [5] L. Zhang, Z. Zhang, R. Siegwart, and J. J. Chung, “Distributed pdop coverage control: Providing large-scale positioning service using a multi-robot system,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 2217–2224, 2021, doi: 10.1109/LRA.2021.3059625.
- [6] C. Yan, Z. Zhang, W. Bao, K. Zhang, and L. Zhang, “Collision-free robust trajectory tracking control for fixed-wing uav with flexible performance constraints,” *Aerospace Science and Technology*, vol. 168, p. 111217, 2026, doi: 10.1016/j.ast.2025.111217.
- [7] Z. Xu, X. Zhang, L. Zhang, Y. Xie, Z. Chen, and D. Wang, “Fault diagnosis and initial alignment of redundant sensors under large misalignment angle,” *IEEE Transactions on Industrial Informatics*, vol. 22, no. 7, pp. 5678–5687, 2026, doi: 10.1109/TII.2026.3670431.
- [8] H. Tan, B. Shen, Q. Li, and H. Liu, “Recursive filtering for stochastic systems with filter-and-forward successive relays,” *IEEE/CAA Journal of Automatica Sinica*, vol. 11, no. 5, pp. 1202–1212, 2024, doi: 10.1109/JAS.2023.124110.
- [9] H. Tan, B. Shen, and J. Kurths, “Set-membership filtering under filter-and-forward successive relay mechanism,” *Automatica*, vol. 193, p. 113206, 2026, doi: 10.1016/j.automatica.2026.113206.
- [10] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, “Do as i can, not as i say: Grounding language in robotic affordances,” in *Proceedings of the 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 287–318. [Online]. Available: <https://proceedings.mlr.press/v205/ichter23a.html>
- [11] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 9493–9500, doi: 10.1109/ICRA48891.2023.10160591.
- [12] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, “Progprompt: Generating situated robot task plans using large language models,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 11 523–11 530, doi: 10.1109/ICRA48891.2023.10161317.
- [13] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “VoxPoser: Composable 3D value maps for robotic manipulation with language models,” in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 540–562. [Online]. Available: <https://proceedings.mlr.press/v229/huang23b.html>
- [14] S. S. Kannan, V. L. N. Venkatesh, and B.-C. Min, “SMART-LLM: Smart multi-agent robot task planning using large language models,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2024, pp. 12 140–12 147, doi: 10.1109/IROS58592.2024.10802322.
- [15] Z. Mandi, S. Jain, and S. Song, “Roco: Dialectic multi-robot collaboration with large language models,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 286–299, doi: 10.1109/ICRA57147.2024.10610855.
- [16] K. Obata, T. Aoki, T. Horii, T. Taniguchi, and T. Nagai, “LiP-LLM: Integrating linear programming and dependency graph with large language models for multi-robot task planning,” *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 1122–1129, 2025, doi: 10.1109/LRA.2024.3518105.
- [17] A. Lykov, S. Karaf, M. Martynov, V. Serpiva, A. Fedoseev, M. Konenkov, and D. Tsetsurukou, “FlockGPT: Guiding UAV flocking with linguistic orchestration,” in *Proc. IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct)*, 2024, pp. 485–488, doi: 10.1109/ISMAR-Adjunct64951.2024.00141.
- [18] W. Ji, H. Chen, M. Chen, G. Zhu, L. Xu, R. Groß, R. Zhou, M. Cao, and S. Zhao, “GenSwarm: Scalable multi-robot code-policy generation and deployment via language models,” *npj Robotics*, vol. 4, p. 5, 2026, doi: 10.1038/s44182-025-00065-w.
- [19] M. Schuck, D. O. Dahanagammaarachchi, B. Sprenger, V. Vyas, S. Zhou, and A. P. Schoellig, “SwarmGPT: Combining large language models with safe motion planning for drone swarm choreography,” *IEEE Robotics and Automation Letters*, vol. 10, no. 11, pp. 12 237–12 244, 2025, doi: 10.1109/LRA.2025.3619745.
- [20] V. Strobel, M. Dorigo, and M. Fritz, “How foundation models will revolutionize robot swarms,” *Science Robotics*, vol. 11, no. 113, p. eadz1543, 2026, doi: 10.1126/scirobotics.adz1543.